

An Execution-Oriented Evaluation of Generate→Validate→Simulate→Rollback Pipelines for LLM-Assisted NetOps: a Confirmatory Study with gpt-5-mini

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered (c1_exec_benchmark_2026_prereg_v1) and executed a paired confirmatory experiment (N=160 vendor-neutral NetOps tasks) comparing a guarded generate→validate→simulate→rollback pipeline against a generation-only baseline using gpt-5-mini. The frozen design used a shared_initial_generation policy (one generated candidate per task reused across conditions) to economize provider calls. Observed outcomes (artifact-verified): baseline actionable = 0/160; guarded actionable = 0/160; paired difference = 0.00; bootstrap 95% CI [0.00, 0.00] (10,000 resamples, seed=1729); McNemar exact two-sided p = 1.00. The shared_initial_generation policy produced a deterministic ceiling effect that prevented evaluation of the preregistered $\geq 30\%$ relative-reduction hypothesis. We release complete SHA-256-verified artifacts (generation outputs, provider_call logs, per-task scoring JSONs, deterministic simulator traces, analysis scripts) and provide an artifact-grounded diagnosis of the design failure plus prescriptive, artifact-traceable remedies (calibration, independent per-condition generation, simulator predicate validation) to enable operationally meaningful follow-on confirmatory tests.

I. INTRODUCTION

Automating NetOps (intent→configuration, change planning, limited self-healing) with large language models (LLMs) promises productivity gains but creates operational-safety obligations: model proposals must avoid producing actionable misconfigurations when enacted. Recent work argues that operational reliability arises from pipelines combining generation, deterministic validation, simulator enactment, and rollback/repair. We preregistered a focused confirmatory test (c1_exec_benchmark_2026_prereg_v1) to answer: does a guarded pipeline (generate→validate→simulate→rollback) reduce the actionable misconfiguration rate by $\geq 30\%$ relative to a generation-only baseline on a representative vendor-neutral NetOps task set? We executed the frozen paired design with gpt-5-mini, archived all artifacts, and report artifact-verified numeric outcomes. The run produced a degenerate null outcome caused by a preregistered cost-saving design choice (shared_initial_generation). Here we (1) report the verified run results; (2) diagnose the design failure with direct artifact pointers; and (3) recommend reproducible remedies for future confirmatory tests.

II. RELATED WORK

Operationalizing LLMs for NetOps requires end-to-end assurance: verifiers and replayable enactment are foundational (e.g., VeriFlow) and recent NetOps/agentic AIOps surveys emphasise contracts, bounded tool use, canary testing, and rollback-aware scoring. Prior LLM→NetOps studies (intent translation, constraint-guided agents, MeshAgent/MNC/Flow-of-Action families, OpsEval/LogEval) argue that static QA is insufficient and that workflow-centred, enactment-aware evaluation is required. Our study implements a preregistered, paired, pipeline-level contrast and publishes end-to-end artifacts so that the verifier, simulator, and statistical analysis are inspectable and deterministic.

III. METHODOLOGY

Preregistration (c1_exec_benchmark_2026_prereg_v1) froze model = gpt-5-mini; adapter = hosted_netops_gvr_v1; confirmatory tasks N = 160 deterministically sampled/augmented from vendor-neutral public corpora using augmentation seeds [42, 123, 999]; paired comparison of two conditions: baseline (generation-only) and guarded (generate→validate→simulate→rollback). Primary estimand: paired difference in actionable misconfiguration proportion ($P_{\text{actionable_baseline}} - P_{\text{actionable_guarded}}$). Analysis: paired bootstrap percentile CI (10,000 resamples, seed=1729) and McNemar exact two-sided test (primary contrast). To limit provider calls the preregistration permitted a shared_initial_generation policy (single generated candidate per task reused across conditions); independent per-condition generation was pre-specified as an alternative but more costly option. Deterministic safety predicate: a simulator adjudicates whether a canonicalized configuration is actionable; this predicate was the preregistered primary outcome. Retry/missingness: provider transient failures retried up to two times; persistent provider failure recorded and handled per preregistration. All raw outputs, provider_call logs, per-task scoring JSONs, simulator traces, and analysis scripts were archived and SHA-256 hashed (execution manifest).

IV. RESULTS

Execution and primary outcomes. Planned episodes = 320 (160 tasks $\times$ 2 conditions). Completed episodes = 320; failed = 0. Because of the shared_initial_generation policy the run used 160 provider model calls (one candidate per task reused across both conditions). Artifact-verified primary numeric results (from analysis artifacts): baseline actionable = 0/160; guarded actionable = 0/160; paired difference = 0.00. Bootstrap 95% CI (10,000 replicates, seed=1729): [0.00, 0.00]. McNemar exact two-sided p = 1.00. Contingency table: n_11 = 160 (both non-actionable), n_10 = 0, n_01 = 0, n_00 = 0. These values are traceable to the archived per-task scoring JSONs; three representative examples (verifiable by the hashes in the execution manifest) are: 1) task-000001 — execution/scoring/task-000001-baseline.json — SHA-256 = e321b84c41233594d2bebc0d01722cade9ab148516fcf13fd9788 — actionable: false; 2) task-000050 — execution/scoring/task-000050-baseline.json — SHA-256 = e4fa1208479e3d51adbdb7147ea951766fda52baefee705d2320d — actionable: false; 3) task-000160 — execution/scoring/task-000160-baseline.json — SHA-256 = 6310438573e09696982a7edd299d12731966266bd66fb147012 — actionable: false. The analysis artifacts (analysis/paired_contingency_table.csv and analysis/paired_difference_figure.svg) and their SHA-256 hashes are included in the execution manifest.

V. DISCUSSION

Interpretation and diagnosis. The run is internally consistent and fully reproducible from archived artifacts, but the pre-registered primary hypothesis ($\geq 30\%$ relative reduction) was rendered unevaluable by a deterministic ceiling effect caused by the `shared_initial_generation` policy: when the single shared candidate for a task is non-actionable under the deterministic simulator, the guarded pipeline has no opportunity to reduce actionable outcomes for that task. Evidence: `provider_call_audit` shows `cache_hit_count` = 0 and model calls used = 160 (one per task) demonstrating shared candidate reuse; the contingency table is degenerate (n_{11} = 160). Thus the null result reflects a design sensitivity rather than evidence that guarded pipelines are ineffective. Practical, artifact-traceable remedies (implementable using the archived scripts and seeds): (A) Calibration discovery: run a small independent per-condition generation discovery sample (deterministic selection of tasks via archived augmentation seeds) to estimate baseline actionability and select tasks with measurable baseline actionables; (B) Independent per-condition generation for confirmatory contrast (one generated candidate per task per condition) to produce discordant outcomes and permit McNemar/bootstrap inference (at cost of doubled provider calls); (C) Simulator predicate audit: human adjudicate a sampled subset of simulator labels to detect permissive or overly conservative semantics (the per-task scoring JSONs and `analysis_log.jsonl` permit this audit without re-execution). Threats to validity: internal validity — `shared_initial_generation` removed treatment variation; construct validity — simulator predicate may not

capture all real deployment failure modes; external validity — single model (gpt-5-mini) and single simulator limit generalisation. Operational implication: confirmatory runs aiming to quantify pipeline value must preserve experimental headroom; cost-saving reuse strategies can mask treatment effects and invalidate primary contrasts.

VI. LIMITATIONS

- Ceiling effect: the preregistered shared_initial_generation (one candidate per task reused across baseline and guarded) produced baseline actionable = 0/160, eliminating headroom and making the preregistered $\geq 30\%$ relative-reduction target unevaluable.
- Single-model and single-policy scope: the experiment used only gpt-5-mini and the exact frozen generation/validation policy; generalization beyond this model and policy is unresolved.
- Construct validity: the deterministic simulator adjudicator abstracts production risk; its actionable predicate may not capture all real deployment failure modes—human adjudication or canary trials are necessary for production claims.
- External validity: no live device enactment or in-production rollout was performed; results apply to the simulator-backed confirmatory design executed here.
- Design sensitivity: the shared-candidate design reduces provider calls but can mask downstream intervention effects; independent per-condition generation should be used when testing pipeline causal effects.

VII. CONCLUSION

We conducted a fully preregistered, reproducible paired evaluation of a guarded generate→validate→simulate→rollback pipeline vs a generation-only baseline on N=160 vendor-neutral NetOps tasks using gpt-5-mini. Observed baseline actionable = 0/160 and guarded actionable = 0/160; paired difference = 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar p = 1.00). The preregistered shared_initial_generation policy produced a deterministic ceiling effect that prevented evaluation of the preregistered ≥30% relative-reduction hypothesis. All primary outputs, per-task scoring JSONs, provider call logs, simulator traces, and analysis scripts are archived with SHA-256 hashes in the execution manifest to enable deterministic verification. The scientific contribution is an artifact-grounded identification of a common confirmatory design failure (shared candidate reuse) that can mask pipeline benefits and the release of concrete, artifact-traceable remedies (calibration, per-condition generation, simulator predicate audit) to enable operationally meaningful follow-ons. We recommend future confirmatory tests adopt independent per-condition generation or a calibrated discovery stage to preserve inferential headroom.

DISCLOSURE STATEMENT

Mandatory Disclosure Statement. Models, orchestration, and autonomy: primary model = gpt-5-mini (hosted API). Adapter/orchestration family: hosted_netops_gvr_v1. Master prompt and autonomy boundary: this run adhered to the Master Prompt v1 — Autonomous CNSM 2026 Research Run; the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved pre-lock infrastructure development. Pre-lock work: infrastructure development and rehearsal occurred pre-lock and were explicitly excluded from empirical evidence. Post-lock autonomy: after the autonomy lock the autonomous system performed literature review, gap selection, preregistration (c1_exec_benchmark_2026_prereg_v1), prompt generation, code generation, model invocation, experiment execution, analysis, peer review, manuscript drafting, and revision without manual human editing of technical content, numeric results, figures, captions, references, or this Disclosure Statement. Preregistration summary: preregistration id c1_exec_benchmark_2026_prereg_v1 froze the paired design, task selection (N=160 with augmentation seeds [42, 123, 999]), the shared_initial_generation policy, model (gpt-5-mini), primary estimand, analysis plan (paired bootstrap 10,000 resamples, seed=1729; McNemar exact test), retry policy, and stopping rules. Execution and artifacts: all raw model outputs, provider_call logs, canonicalized configurations, per-task scoring JSONs (execution/scoring/*.json), deterministic simulator schema, analysis scripts, and the execution manifest are archived and SHA-256 hashed. Representative artifact hashes (from execution manifest): execution/raw_results.jsonl = 2445d0f8e9be9cfa34ede1bbe80b82d6f05b51bcafc7c1688d0ed8e1c73d8044; analysis/paired_contingency_table.csv = fba3b0ca21c7c7c380ed1134b4fce48b0870bf72a7be0fd8610bf7e6b63a970b; execution/scoring/task-000001-baseline.json = e321b84c41233594d2bec0d01722cade9ab148516fcf13fd9788c94fb638e59; execution/scoring/task-000050-baseline.json = e4fa1208479e3d51adbd7147ea951766fda52bafee705d2320da8870395d90; execution/scoring/task-000160-baseline.json = 6310438573e09696982a7edd299d12731966266bd66fb1470123f0b1549ba8ef. Provider calls and retries: planned maximum model calls = 320; actual provider model calls used = 160 due to the preregistered shared_initial_generation design; transient provider retries permitted up to two attempts per call (no material impact on final confirmatory counts). Human involvement: the Research Director provided topic selection and pre-lock approvals; after autonomy lock no human manually edited the manuscript's technical content or the released numeric artifacts. Reproducibility instruction: to reproduce the analysis deterministically, use the execution manifest file path (execution/execution_manifest.json) and the archived artifact files enumerated therein; validate file integrity by computing SHA-256 and comparing to the listed hashes (example: analysis/paired_contingency_table.csv hash fba3b0ca21c7c7c380ed1134b4fce48b0870bf72a7be0fd8610bf7e6b63a970b).

REFERENCES

- [1] <https://doi.org/10.1145/2342441.2342452>
- [2] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369
- [3] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026
- [4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.34726/12420</div>
- [5] Adepegba Akindayomi Akintade, "From Detection to Verified Action: Operational Readiness for AI-Enabled Cloud Failure Management", 2026, 10.22214/ijraset.2026.84530
- [6] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742
- [7] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567